Phase #2 - Front End Design Document

Overview:

This document outlines the front-end design for the Sentinel Quality Group game distribution service, providing comprehensive class and method specifications alongside corresponding UML diagrams. Functioning as a point-of-sale terminal, the frontend system reads input transaction codes from the console and executes dedicated handler logic within the active session context. Interdependent objects manage session state, user accounts, and product listings throughout the session. Upon completion of each valid transaction, an entry is logged in the session history; during the logout procedure, a file management method adds these entries into a daily transaction file.

Inputs and outputs:

Input/Output	Format	Where it is Handled
Console Input	Transaction code	FrontEnd::run and FrontEnd::Prompt
Console Output	Prompts, Confirmations, and any error messages	In the Frontend handler methods (handleLogin, handleSell, handleBuy, etc)
Current User accounts file	Lines containing username, type, and credits with a min of 28 characters.	UserAccouts::load
Available games file	Lines containing game name, seller, and price.	GameCatalog::Load
Daily Transaction File	A formatted line per transaction, with the last line being the 00 end-of-session record	TransactionLog methods

Component Type	Name	Description

Namespace	Format	Provides helpers for reading and writing fields, used by the data file.
Function	Format::trim	Removes leading and trailing whitespaces from an inputted string.
Function	Format::toLower	Converts all characters in a string to lowercase.
Function	Format::padRight	Pads a string on the right with a specific width.
Function	Format::formatMoney	Formats an integer amount of money into a string with leading 0's.
Function	Format::parseMoney	Parses a string into an integer representing cents and returns success status.
Class	FrontEnd	Reads transactions one line at a time and applies them to the accounts and games.
Method	FrontEnd::FrontEnd (Constructor)	Initializes streams and file paths for the frontend interface.
Method	FrontEnd::run	Starts the main application loop, reading and processing user input.
Method	FrontEnd::prompt	Prompts a user with a message and reads their response.
Method	FrontEnd::processTransaction	Processes a given transaction code and routes it to the appropriate handler.

Method	FrontEnd::handleHelp	Displays the list of available transactions based on the user's role and login status.
Method	FrontEnd::handleLogin	Handles the user login process by validating credentials and loading data.
Method	FrontEnd::handleLogout	Handles logout function and writes session logs to file.
Method	FrontEnd::handleCreate	Handles creation of new accounts (admins only).
Method	FrontEnd::handleDelete	Handles deletion of accounts (admins only).
Method	FrontEnd::handleSell	Handles selling of games and putting up listings.
Method	FrontEnd::handleBuy	Handles purchasing a game from a seller.
Method	FrontEnd::handleRefund	Handles giving out refunds (admin only).
Method	FrontEnd::handleAddCredit	Handles adding credit to a user account.
Class	GameCatalog	Manages available games read from file.
Method	GameCatalog::load	Loads the game catalog from a text file.
Method	GameCatalog::findGame	Finds and returns a pointer to a game listing by its name.

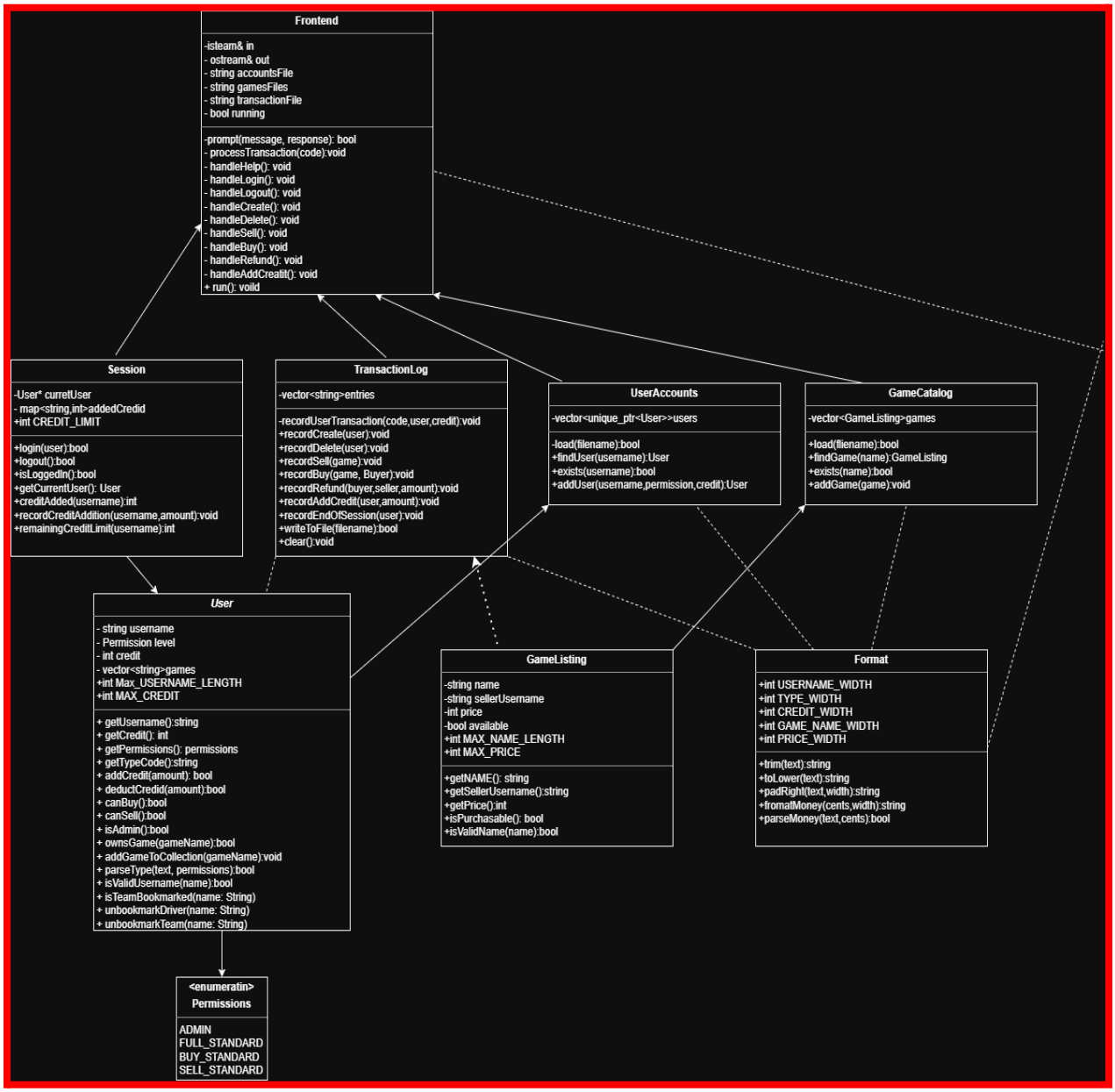
Method	GameCatalog::exists	Checks if a game exists in the catalog.
Method	GameCatalog::addGame	Adds a new game to the catalog collection.
Class	GameListing	Defines a game listing with its respective data and attributes.
Method	GameListing::GameListing (Constructor)	Initializes a game listing with a name, seller, price, and availability.
Method	GameListing::getName	Returns the name of the game.
Method	GameListing::getSellerUsername	Returns the seller's username.
Method	GameListing::getPrice	Returns the price of the game.
Method	GameListing::isPurchasable	Checks if a game is available to purchase.
Method	GameListing::isValidName	Validates if a given name meets the length and non-empty criteria.
Function	main	Initializes and runs the Steam clone front-end interface.
Class	Session	Tracks the current logged-in user and session credit additions.
Method	Session::login	Logs a user in if no other user is logged in.
Method	Session::logout	Logs out the current user and clears session state.

Method	Session::isLoggedIn	Checks whether a user is currently logged in.
Method	Session::getCurrentUser	Returns a pointer to the currently logged-in user.
Method	Session::creditAdded	Returns the amount of credit added to a username during this session.
Method	Session::recordCreditAddition	Records credit added to an account in the session.
Method	Session::remainingCreditLimit	Returns the remaining session credit limit for a user.
Class	TransactionLog	Collects transactions over a session and builds the daily transaction file.
Method	TransactionLog::recordUserTransaction	Formats and appends a generic user transaction entry.
Method	TransactionLog::recordCreate	Records a user creation transaction.
Method	TransactionLog::recordDelete	Records a user deletion transaction.
Method	TransactionLog::recordSell	Records a game selling transaction.
Method	TransactionLog::recordBuy	Records a game purchasing transaction.
Method	TransactionLog::recordRefund	Records a refund transaction between buyer and seller.
Method	TransactionLog::recordAddCredit	Records a credit addition transaction.

Method	TransactionLog::recordEndOfSession	Records the end of session transaction entry.
Method	TransactionLog::writeToFile	Appends collected transaction entries to a file.
Method	TransactionLog::clear	Clears all recorded transaction entries.
Class	User	Defines individual user accounts, permissions, and owned games.
Method	User::User (Constructor)	Initializes a user with a username, permissions, and credit balance.
Method	User::getUsername	Returns the user's username.
Method	User::getCredit	Returns the user's credit balance.
Method	User::getPermissions	Returns the user's permission level.
Method	User::getTypeCode	Returns the string type code for the user permissions.
Method	User::addCredit	Adds credit to the user account if within limits.
Method	User::deductCredit	Deducts credit from the user account if sufficient funds exist.
Method	User::canBuy	Checks if the user permissions allow buying games.
Method	User::canSell	Checks if the user permissions allow selling games.

Method	User::isAdmin	Checks if the user is an administrator.
Method	User::ownsGame	Checks if a game is already in the user's collection.
Method	User::addGameToCollection	Adds a game name to the user's collection.
Method	User::parseType	Parses input text to determine the user permission level.
Method	User::isValidUsername	Validates if a username meets length and non-empty criteria.
Class	UserAccounts	Manages the set of user accounts read from the current user accounts file.
Method	UserAccounts::load	Replaces all accounts with the file's contents.
Method	UserAccounts::findUser	Finds and returns a pointer to a user by their username.
Method	UserAccounts::exists	Checks if a user with a specific username exists.
Method	UserAccounts::addUser	Adds a new user account to the collection with specified permissions and credits.

UML



https://drive.google.com/file/d/1ZwmqJfqW_6tStmFGp812mAaJyhtqoRJj/view?usp=sharing

AI Usage Disclaimer

https://claude.ai/share/1152a747-c1d2-4387-aca7-d5f8fa54e163

https://share.gemini.google/lypEXND0Cugw - Talha Fatiwala